St. Francis Institute of Technology, Mumbai- 400 103
Department Of Information Technology
A.Y. 2024-2025
Class: BE-ITA/B, Semester: VIII
Subject: BlockChain Lab

Experiment –2

1. **Aim**: To implement smart contract using Remix IDE

2. **Objective**: To …
   i. explain what smart contracts are.
   ii. understand coding smart contracts using solidity language.
   iii. test a smart contract by using Remix IDE

3. **Lab outcome**: After performing the experiment, the students will be able to implement smart contracts in Ethereum using different development frameworks (PO3, PSO2, BL3)

4. **Prerequisite**:
   i. Fundamental knowledge of blockchain
   ii. Knowledge of the Ethereum platform and Remix IDE
   iii. Familiarity with the Solidity programming language

5. **Requirements**: The following are the requirements:
   i. Remix IDX

6. **Pre-Experiment Theory:**

   What is Blockchain?
   Blockchain is a shared, immutable ledger that facilitates the process of recording transactions and tracking assets in a business network. An asset can be tangible (a house, car, cash, land) or intangible (intellectual property, patents, copyrights, branding). Virtually anything of value can be tracked and traded on a blockchain network, reducing risk and cutting costs for all involved.

   What is Ethereum?
   Ethereum is a decentralized blockchain designed to be highly secure, fault-tolerant, and programmable Ethereum blockchain is a choice for many developers and businesses. As said programmable, the main task of Ethereum is to securely execute and verify the application code known as smart contracts. Ethereum helps to build native scripting languages(solidity) and EVM. Ethereum consensus mechanism is proof of work to operate to verify the new transaction

   What is Remix IDE?
   It is an online IDE for creating solid, smart contracts, so you do not need to install or download anything to do any setup. You can develop, deploy, and administer your solidity smart contract using Remix IDE.

What is Solidity?
Solidity is a statically typed, contract-oriented, high-level language for implementing smart contracts that run on the Ethereum Virtual Machine. Smart contracts are programs that are executed inside a peer- to-peer network where nobody has special authority over the execution, and thus they allow anyone to implement tokens of value, ownership, voting, and other kinds of logic.

What is smart contract?
A smart contract is a small program that runs on an Ethereum blockchain. Once the smart contract is deployed on the Ethereum blockchain, it cannot be changed. To deploy the smart contract to Ethereum, you must pay the ether (ETH) cost. Understand it as a digital agreement that builds trust and allows both parties to agree on a particular set of conditions that cannot be tampered with.

7. Laboratory Exercise

A. Steps to be implemented.

To Follow the procedure given below to build smart contract in Remix IDE
1. Open Remix IDE in Google Chrome
2. Write smart contract by creating new file under contract folder, with .sol extension.
3. Write your contract code using solidity language.
4. Click on Solidity compiler icon.
5. Choose compiler version or keep default and click on compile button.
6. Click on Deploy and Run Transaction icon.
7. Choose Environment Remix VM (London). and choose Account Number or keep default.
8. Click on deploy button to deploy smart contract on Ethereum blockchain.
9. Under deployed contract, get the output of your contract.

B. Program Code

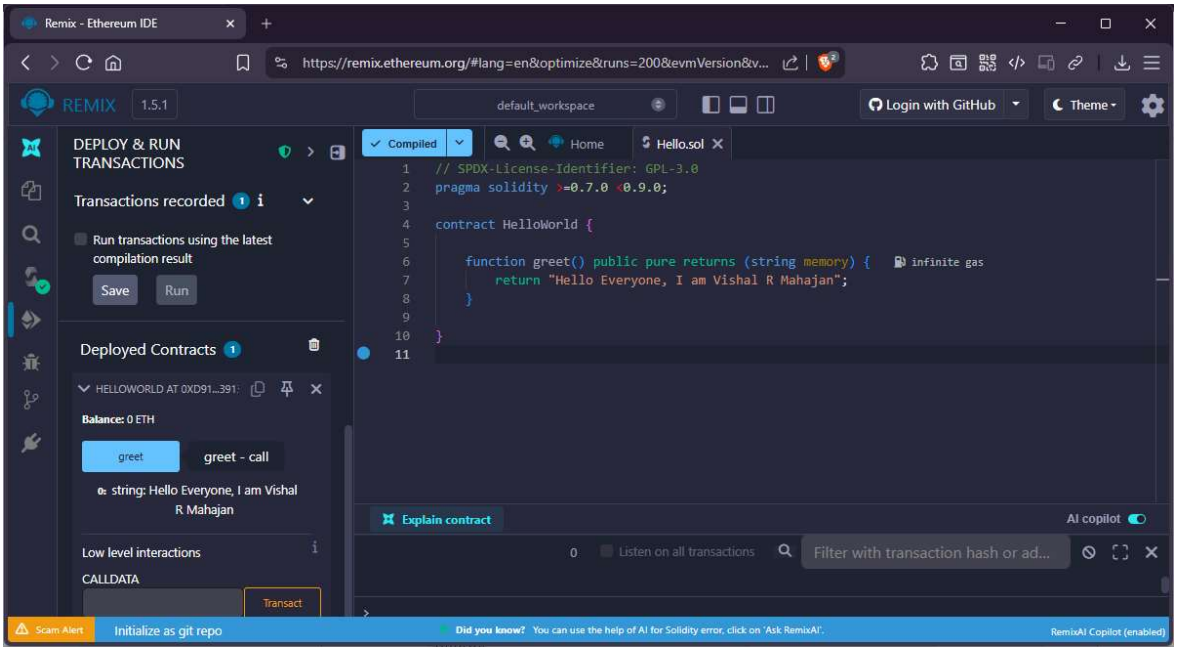
1. Write the first smart contract HelloWorld.sol as follows.

```
==========================
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract HelloWorld {

   function greet() public pure returns (string memory) {
      return "Hello Everyone, I am Vishal R Mahajan";
   }

}
==========================
```



2. Write a smart contract storage.sol to store a number and to retrieve the number as follows.

==========

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract Storage1 {

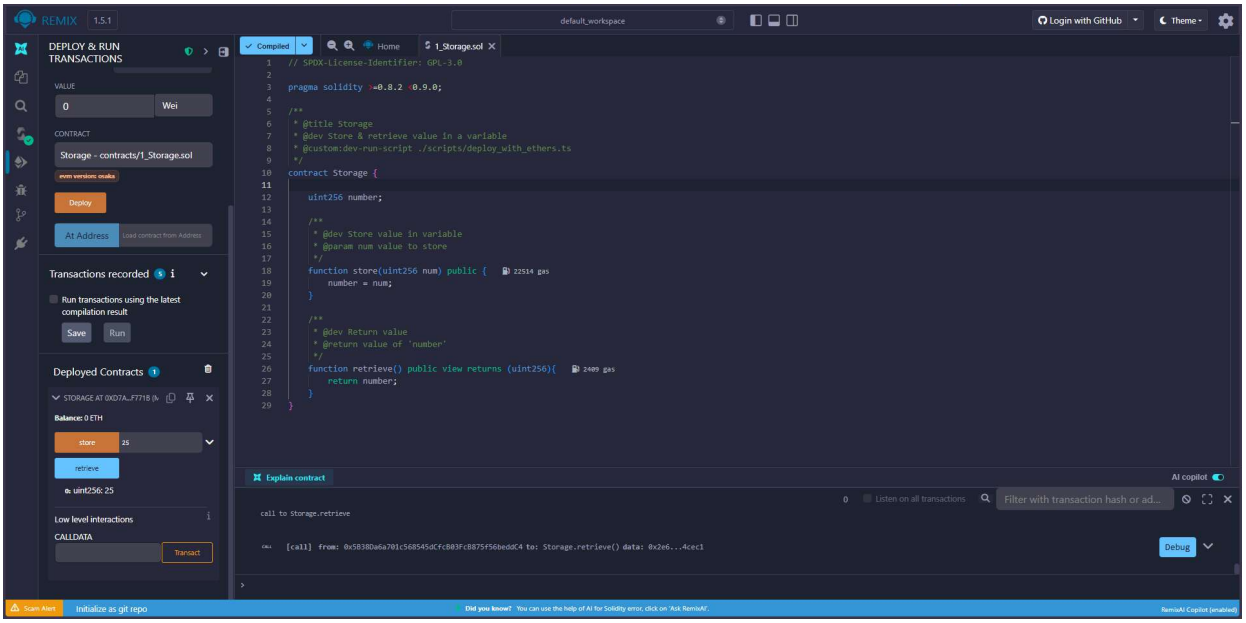
    uint public number;

    function storeInt(uint _num) public {
        number = _num;
    }

    function retrieve() public view returns (uint) {
        return number;
    }

}
```

==========



3. Write a smart contract employee.sol to create an employee database with fields like empId, empName, empDept, empDesignation. Create functions to addEmployee and getEmployee. Use concept of structure.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract EmployeeDatabase {

    // Structure to store employee details
    struct Employee {
        uint empId;
        string empName;
        string empDept;
        string empDesignation;
    }

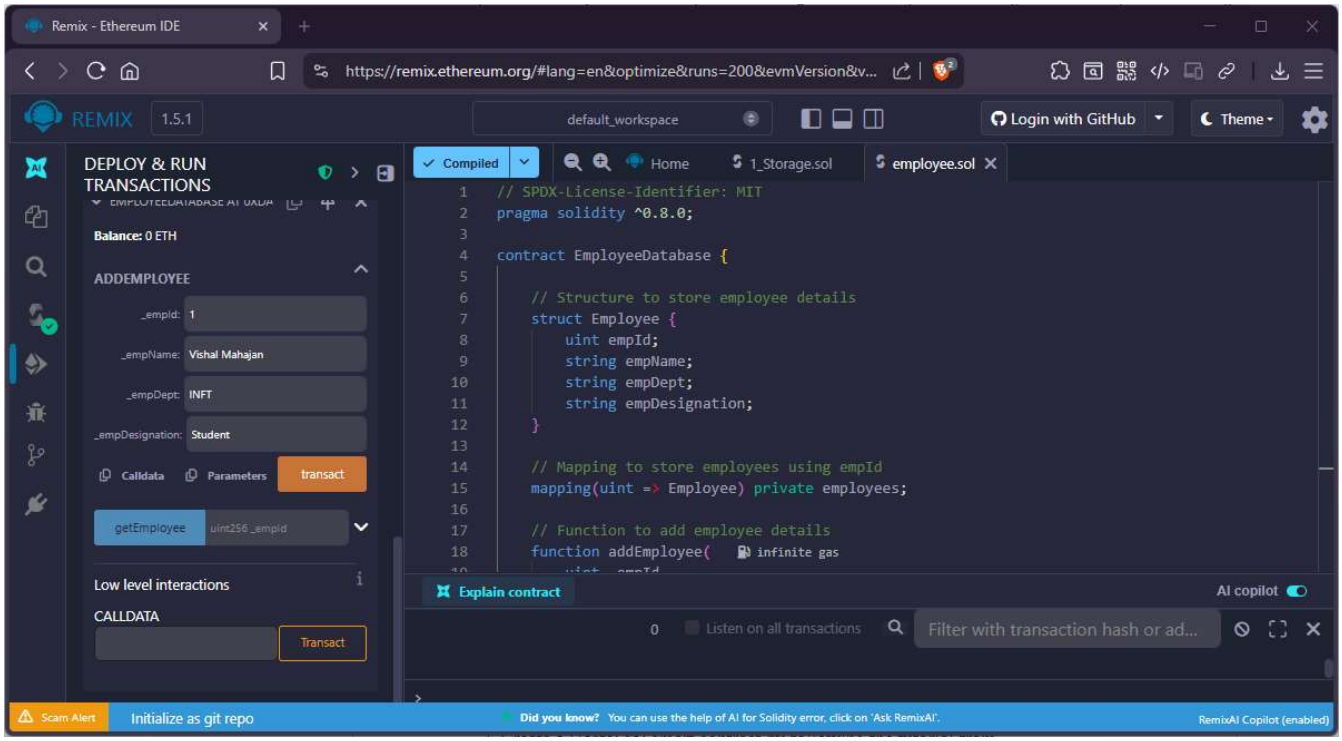
    // Mapping to store employees using
empId
    mapping(uint => Employee) private
employees;

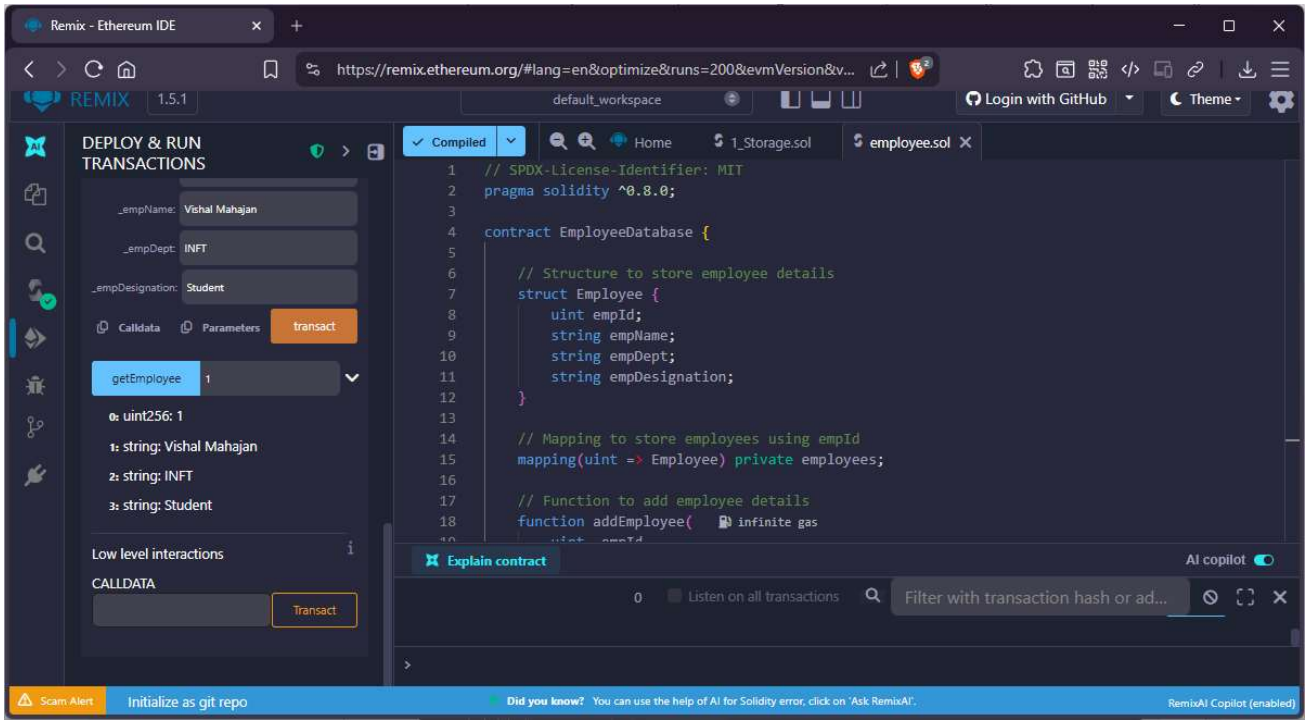
    // Function to add employee details
    function addEmployee(
        uint _empId,
        string memory _empName,
        string memory _empDept,
        string memory _empDesignation
    ) public {

        employees[_empId] = Employee({
            empId: _empId,
            empName: _empName,
            empDept: _empDept,
            empDesignation:
_empDesignation
        });
    }

// Function to retrieve employee details
    function getEmployee(uint _empId)
        public
        view
        returns (
            uint,
            string memory,
            string memory,
            string memory
        )
    {
        Employee memory emp =
employees[_empId];

        return (
            emp.empId,
            emp.empName,
            emp.empDept,
            emp.empDesignation
        );
    }
}
```





8. Post Experimental Exercise-

C. Questions:

1. Write down key properties and advantages of smart contracts
2. Create a Hostel.sol smart contract by following the tutorial from https://www.loginradius.com/blog/engineering/guest-post/ethereum-smart-contract-tutorial

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Hostel {

    address public warden;
    uint public rooms;
    uint public occupied;

    constructor(uint _rooms) {
        warden = msg.sender;
        rooms = _rooms;
    }

    // Structure to store student details
    struct Student {
        string name;
        bool inside;
        uint complaints;
    }

    // Mapping of student address to student details
    mapping(address => Student) public students;

    // Events
    event CheckedIn(address student);
    event Complaint(address student);
    event CheckedOut(address student);

    // Function for student check-in
    function checkIn(string memory _name) public {
        require(!students[msg.sender].inside, "Already inside");
        require(occupied < rooms, "Hostel full");

        students[msg.sender] = Student(_name, true, 0);
        occupied++;

        emit CheckedIn(msg.sender);
    }

    // Function to register a complaint
    function complain() public {
        require(students[msg.sender].inside, "Not in hostel");

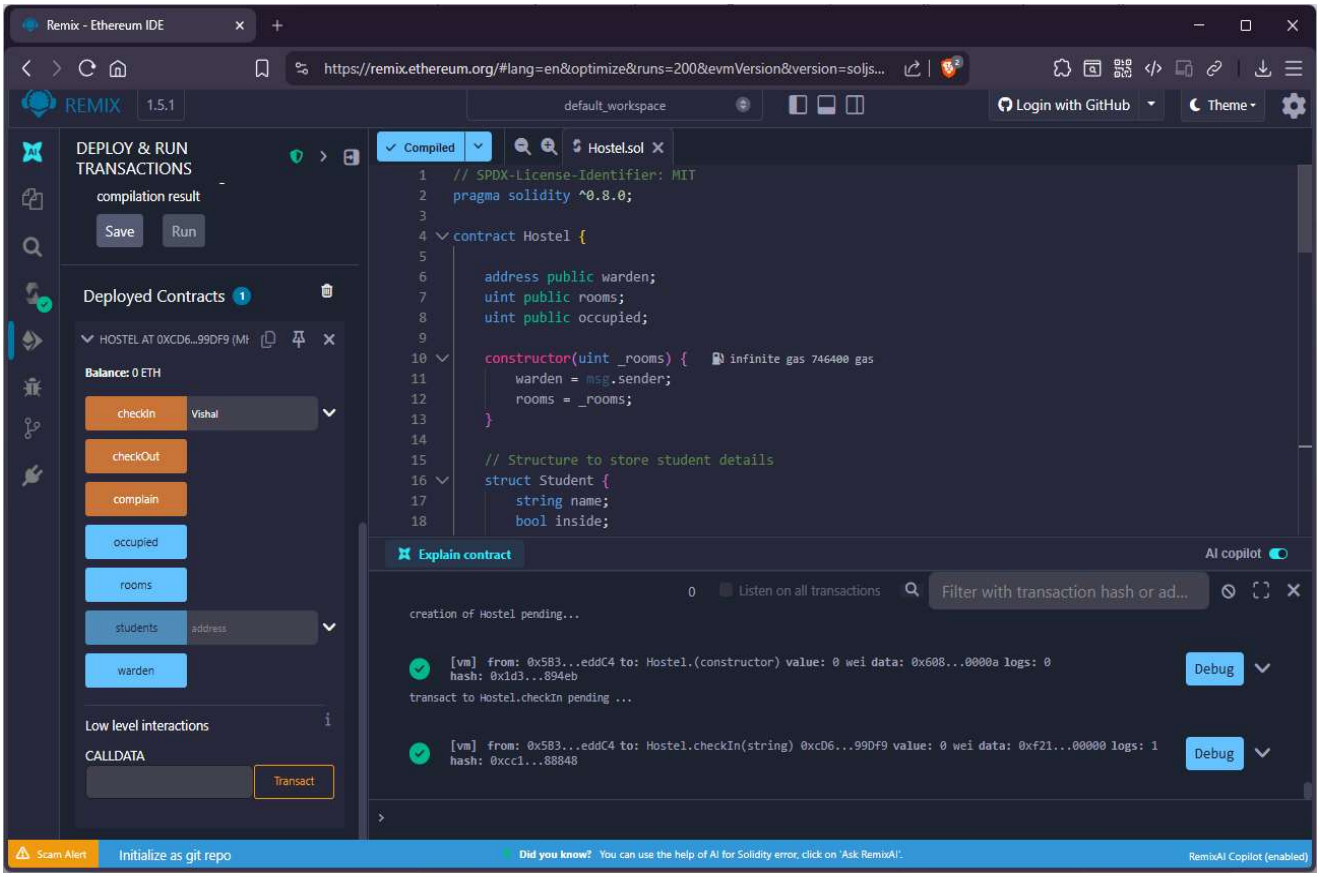
        students[msg.sender].complaints++;

        emit Complaint(msg.sender);
    }

    // Function for student check-out
    function checkOut() public {
        require(students[msg.sender].inside, "Not inside");

        students[msg.sender].inside = false;
        occupied--;

        emit CheckedOut(msg.sender);
    }
}
```



D. Results/Observations/Program output:
Present the program input/output results if any and comment on the same.

E.Conclusion:
1. Write what was performed in the experiment
2. Write which tools you used to perform the experiment
3. Write what you inferred from the output obtained

F. References:
[1] https://www.simplilearn.com/tutorials/blockchain-tutorial/what-is-smart-contract
[2] https://ethereum.org/en Mastering Ethereum, Building Smart Contract and Dapps, Andreas M. Antonopoulos Dr. Gavin Wood, O'reil